SIMATS ENGINEERING

ASSESSMENT TOOL 3 - LOW (INDUSTRY PROBLEM SOLVING TASKS)

COURSE CODE: CSA09 COURSE NAME: Programming in Java

COURSE OUTCOME COVERED
CO4: *Design and develop GUI-based user interfaces using Abstract Window Toolkit, Swing, and Applets by implementing event handling, layout managers, AWT controls, and menus. (BL3, BL5)*

Assessment Tool Weightage: 10%

QUESTIONS: 5 Total Marks: 50

ANNEXURE A
INDUSTRY PROBLEM-SOLVING TASKS

Code of Conduct:

I K.GNANESWAR KUMAR - (192572105) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Rubrics for Evaluation:

Criteria	Underst an ding of the Industry Problem	Applicati on of Engineeri ng Concepts	Soluti on Desig n	Use of Tools	Analyzing	Professio nalism	Total (25%)	Total (100)
Weig hta ge	20%	25%	25%	15%	10%	5%		
Q1								
Q2								
Q3								
Q4								

Q5								
Total								

ANNEXURE

INDUSTRY PROBLEM SOLVING TASKS

1. Hospital Patient Billing System

Develop a Java program for a hospital billing system using Class and Objects. Data Members:

- Patient ID
- Patient Name
- Room Type (General/Special)
- Number of Days Admitted
- Treatment Charges

Methods:

1. Read patient details.
2. Calculate room charges based on room type.
3. Calculate the total hospital bill.
4. Display patient billing details.

Note: General Room = ₹1,500/day, Special Room = ₹3,000/day.

2. Airline Ticket Reservation System

Develop a Java program for an airline reservation system using Class and Objects.
Data Members:

- Passenger ID
- Passenger Name
- Flight Number
- Ticket Fare
- Number of Tickets

Methods:

1. Read passenger details.
2. Calculate total ticket cost.
3. Apply a 5% discount if more than 5 tickets are booked.
4. Display booking and payment details.

Note: GST = 12% on the final amount.

3. Vehicle Rental Management System

Develop a Java program for a vehicle rental company using Class and Objects.
Data Members:

- Customer ID
- Customer Name
- Vehicle Type (Car/Bike)
- Number of Rental Days

Methods:

1. Read customer details.

2. Calculate rental charges.
3. Add a late return penalty if applicable.
4. Display the final rental bill.

Note: Car = ₹2,000/day, Bike = ₹500/day.

4. Manufacturing Production Monitoring System

Develop a Java program to monitor daily production in a manufacturing company using Class and Objects.

Data Members:

- Product ID
- Product Name
- Target Quantity
- Produced Quantity

Methods:

1. Read product details.
2. Compare actual production with the target.
3. Calculate production efficiency percentage.
4. Display production status (Achieved/Not Achieved).

Note: Efficiency = (Produced Quantity / Target Quantity) × 100.

5. Online Course Enrollment System

Develop a Java program for an online learning platform using Class and Objects.

Data Members:

- Student ID
- Student Name
- Course Name
- Course Fee

Methods:

1. Read student and course details.
2. Calculate the payable amount.
3. Provide a 15% scholarship discount if the fee exceeds ₹20,000.
4. Display enrollment and payment details.

Note: Registration fee = ₹500.

ANSWERS:

1. Hospital Patient Billing System

```
import java.util.Scanner;

class Patient {
  int patientId;
  String patientName;
  String roomType;
  int daysAdmitted;
  double treatmentCharges;

  void readDetails() {
     Scanner sc = new Scanner(System.in);

     System.out.print("Enter Patient ID: ");
     patientId = sc.nextInt();
     sc.nextLine();

     System.out.print("Enter Patient Name: ");
     patientName = sc.nextLine();

     System.out.print("Enter Room Type (General/Special): ");
     roomType = sc.nextLine();

     System.out.print("Enter Number of Days Admitted: ");
     daysAdmitted = sc.nextInt();

     System.out.print("Enter Treatment Charges: ");
     treatmentCharges = sc.nextDouble();
  }

  double calculateRoomCharges() {
     if (roomType.equalsIgnoreCase("General"))
        return daysAdmitted * 1500;
     else
        return daysAdmitted * 3000;
  }

  double calculateTotalBill() {
     return calculateRoomCharges() + treatmentCharges;
  }

  void displayDetails() {
```

```
    System.out.println("\nPatient ID: " + patientId);
    System.out.println("Patient Name: " + patientName);
    System.out.println("Room Charges: ₹" + calculateRoomCharges());
    System.out.println("Treatment Charges: ₹" + treatmentCharges);
    System.out.println("Total Bill: ₹" + calculateTotalBill());
  }

  public static void main(String[] args) {
    Patient p = new Patient();
    p.readDetails();
    p.displayDetails();
  }
}
```

---

2. Airline Ticket Reservation System

```
import java.util.Scanner;

class AirlineReservation {
  int passengerId;
  String passengerName;
  String flightNumber;
  double ticketFare;
  int numberOfTickets;

  void readDetails() {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter Passenger ID: ");
    passengerId = sc.nextInt();
    sc.nextLine();

    System.out.print("Enter Passenger Name: ");
    passengerName = sc.nextLine();

    System.out.print("Enter Flight Number: ");
    flightNumber = sc.nextLine();

    System.out.print("Enter Ticket Fare: ");
    ticketFare = sc.nextDouble();

    System.out.print("Enter Number of Tickets: ");
    numberOfTickets = sc.nextInt();
  }
```

```
  double calculateTotalCost() {
    double total = ticketFare * numberOfTickets;

    if (numberOfTickets > 5)
      total = total - (total * 0.05);

    double gst = total * 0.12;
    return total + gst;
  }

  void displayDetails() {
    System.out.println("\nPassenger ID: " + passengerId);
    System.out.println("Passenger Name: " + passengerName);
    System.out.println("Flight Number: " + flightNumber);
    System.out.println("Total Amount (with GST): ₹" + calculateTotalCost());
  }

  public static void main(String[] args) {
    AirlineReservation a = new AirlineReservation();
    a.readDetails();
    a.displayDetails();
  }
}
```

---

3. Vehicle Rental Management System

```
import java.util.Scanner;

class VehicleRental {
  int customerId;
  String customerName;
  String vehicleType;
  int rentalDays;
  double penalty;

  void readDetails() {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter Customer ID: ");
    customerId = sc.nextInt();
    sc.nextLine();

    System.out.print("Enter Customer Name: ");
```

```
    customerName = sc.nextLine();

    System.out.print("Enter Vehicle Type (Car/Bike): ");
    vehicleType = sc.nextLine();

    System.out.print("Enter Rental Days: ");
    rentalDays = sc.nextInt();

    System.out.print("Enter Late Return Penalty: ");
    penalty = sc.nextDouble();
  }

  double calculateRentalCharges() {
    if (vehicleType.equalsIgnoreCase("Car"))
      return rentalDays * 2000;
    else
      return rentalDays * 500;
  }

  double finalBill() {
    return calculateRentalCharges() + penalty;
  }

  void displayDetails() {
    System.out.println("\nCustomer ID: " + customerId);
    System.out.println("Customer Name: " + customerName);
    System.out.println("Rental Charges: ₹" + calculateRentalCharges());
    System.out.println("Penalty: ₹" + penalty);
    System.out.println("Final Bill: ₹" + finalBill());
  }

  public static void main(String[] args) {
    VehicleRental v = new VehicleRental();
    v.readDetails();
    v.displayDetails();
  }
}
```

---

4. Manufacturing Production Monitoring System

```
import java.util.Scanner;

class Production {
  int productId;
```

```java
    String productName;
    int targetQuantity;
    int producedQuantity;

    void readDetails() {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Product ID: ");
        productId = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Product Name: ");
        productName = sc.nextLine();

        System.out.print("Enter Target Quantity: ");
        targetQuantity = sc.nextInt();

        System.out.print("Enter Produced Quantity: ");
        producedQuantity = sc.nextInt();
    }

    double calculateEfficiency() {
        return ((double) producedQuantity / targetQuantity) * 100;
    }

    void displayStatus() {
        System.out.println("\nProduct ID: " + productId);
        System.out.println("Product Name: " + productName);
        System.out.println("Efficiency: " + calculateEfficiency() + "%");

        if (producedQuantity >= targetQuantity)
            System.out.println("Status: Achieved");
        else
            System.out.println("Status: Not Achieved");
    }

    public static void main(String[] args) {
        Production p = new Production();
        p.readDetails();
        p.displayStatus();
    }
}
```

---

5. Online Course Enrollment System

```
import java.util.Scanner;

class CourseEnrollment {
  int studentId;
  String studentName;
  String courseName;
  double courseFee;

  void readDetails() {
     Scanner sc = new Scanner(System.in);

     System.out.print("Enter Student ID: ");
     studentId = sc.nextInt();
     sc.nextLine();

     System.out.print("Enter Student Name: ");
     studentName = sc.nextLine();

     System.out.print("Enter Course Name: ");
     courseName = sc.nextLine();

     System.out.print("Enter Course Fee: ");
     courseFee = sc.nextDouble();
  }

  double calculatePayableAmount() {
     double fee = courseFee;

     if (courseFee > 20000)
        fee = fee - (courseFee * 0.15);

     return fee + 500; // Registration Fee
  }

  void displayDetails() {
     System.out.println("\nStudent ID: " + studentId);
     System.out.println("Student Name: " + studentName);
     System.out.println("Course Name: " + courseName);
     System.out.println("Payable Amount: ₹" + calculatePayableAmount());
  }

  public static void main(String[] args) {
```

```
        CourseEnrollment c = new CourseEnrollment();
        c.readDetails();
        c.displayDetails();
    }
}
```

RUBRICS FOR CALCULATION:

Criteria	**Weigh tage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Develop in g**	**Level 1 – Beginning**
Criteria	Weig ht age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understan din g of Industry Problem	20%	Comprehensi ve understandin g of real-world problem and constraints	Good understandi ng with minor gaps in requirements	Basic understa n ding; some requirem e nts unclear	Poor understandin g; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited applicati on of concepts	Incorrect or no application of concepts

Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementa tio n	Limited or inconsis te nt use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professiona lis m	5%	Highly professional, well documented, and clearly presented work	Good documentati on and presentation	Basic docume nta tion with some gaps	Poor documentati on and presentation